

Лабораторная работа №6

Интеграция WPF-приложения с ASP.NET Core Web API.

Содержание

1. Цель работы 🎯	3
2. Общая информация 📄	4
2.1. Роли каждого проекта	4
2.2. Изменения в коде	5
3. Реализация клиентского сервиса (ApiService.cs) ⚙️	6
4. Реализация MainViewModel.cs 🗄️	9
5. OrderViewModel.cs — управление заказами 🛒	14
6. Контрольные вопросы ❓	17

List of Listings

3.1	Клиентского сервиса	7
4.1	MainViewModel.cs	12
5.1	OrderViewModel.cs	16

1. Цель работы

Цель: организовать взаимодействие между клиентским WPF-приложением и серверным REST API, обеспечив обмен данными через HTTP-запросы.



Компонент	Назначение
StoreManagerApi	Серверная часть, работа с БД
StoreManager_4lab	Клиентское WPF-приложение
SQLite БД	Хранение данных в store.db

Рис. 1.1: Структура проекта

Внимание: проект ASP.net созданный в лабораторной №5 изменять в ходе данной лабораторной не нужно будет. Необходимо привести проект созданный в лабораторной №4 к тому виду, который будет рассмотрен в данной лабораторной и убедиться, что все функции (кнопки) выполняются без ошибок.

2. Общая информация

1. WPF-приложение — это клиент, т.е. пользовательский интерфейс, установленный на компьютере пользователя.
2. ASP.NET Web API — это сервер, который обрабатывает запросы, работает с базой данных и отправляет данные клиенту (например, в формате JSON).

2.1. Роли каждого проекта

ASP.NET (сервер):

1. Отвечает за доступ к базе данных.
2. Использует DbContext из Entity Framework, чтобы читать и сохранять данные.
3. Предоставляет HTTP API (например, GET, POST, PUT, DELETE), к которым можно обращаться через интернет или локальную сеть.

WPF (клиент):

1. Отображает данные пользователю.
2. Отправляет HTTP-запросы к ASP.NET API, чтобы получить или отправить данные.
3. Не подключается к базе данных напрямую, а работает через API. Поэтому DbContext здесь не нужен.

Компонент	Лабораторная №4 (локальная БД)	Лабораторная №6 (через API)
Хранение данных	SQLite через EntityFrameworkCore прямо в WPF-приложении	ASP.NET Core Web API, клиент обращается к серверу
Репозитории	Прямой доступ к DbContext	Используются HttpClient и ApiService
Работа с данными	StoreDbContext и LINQ-запросы	REST-запросы: GET, POST, PUT, DELETE
Добавление / удаление / редактирование	Прямое обращение к базе через EF	Отправка данных в контроллеры Web API
Обновление интерфейса	Автоматически при изменении коллекций	После получения данных через HTTP-запрос

Таблица 1 – Архитектурные изменения

2.2. Изменения в коде

Контекст базы данных (DbContext)

1. Было: EF Core DbContext создавался и использовался прямо в MainViewModel и других ViewModel.
2. Стало: База данных вынесена на сервер в ASP.NET Core Web API, клиент WPF не работает с EF напрямую.

Причина изменения: разделение ответственности — клиент занимается только отображением, сервер — бизнес-логикой и хранением.

В данной таблице приведены ключевые различия в подходе к работе с данными между локальным доступом через DbContext (в лабораторной №4) и использованием Web API через HttpClient (в лабораторной №6).

Описание операции	Лабораторная №4 (локально)	Лабораторная №6 (через Web API)	Изменение
Добавление товара	<code>_context.Products.Add(product);</code>	<code>await_api.AddProductAsync(product);</code>	Используется API вместо EF Core
Сохранение изменений	<code>_context.SaveChanges();</code>	Выполняется внутри метода API	Уходит внутрь ASP.NET API
Загрузка товаров	<code>foreach(var pin_context.Products)</code>	<code>var products = await_api.GetProductsAsync();</code>	Получение данных по HTTP
Создание заказа	<code>_context.Orders.Add(order);</code>	<code>await_api.AddOrderAsync(order);</code>	Заказ создается через API
Удаление заказа/товара	<code>_context.Orders.Remove(order);</code>	<code>await _api.DeleteOrderAsync(order.Id);</code>	Операция выполняется на сервере

Таблица 2 – Основные изменения в методах используемых в приложении

Таким образом, ключевым отличием лабораторной №6 стало отделение клиентской логики (WPF) от серверной (ASP.NET Core Web API) и переход от прямого доступа к базе данных к REST-взаимодействию через HTTP-запросы.

3. Реализация клиентского сервиса (ApiService.cs) ⚙️

ApiService.cs — клиентский сервис доступа к Web API. Класс ApiService реализует клиентскую часть взаимодействия WPF-приложения с серверным Web API. Он инкапсулирует вызовы HTTP-запросов к контроллерам товаров и заказов, используя HttpClient. Это позволяет централизованно управлять всеми запросами к серверу и облегчает сопровождение кода.

```
1 namespace StoreManager_6lab.Services
2 {
3     public class ApiService
4     {
5         private readonly HttpClient _http; // HTTP-клиент для запросов к API
6
7         public ApiService()
8         {
9             var handler = new HttpClientHandler
10             {
11                 ServerCertificateCustomValidationCallback = (message, cert, chain, errors) => true
12                 ↪ // Разрешить самоподписанные сертификаты
13             };
14
15             _http = new HttpClient(handler)
16             {
17                 BaseAddress = TryUseHttps()
18                 ? new Uri("https://localhost:7259/api/")
19                 : new Uri("http://localhost:5107/api/") // Базовый адрес сервера API
20             };
21
22             // Попытка проверить доступность HTTPS API
23             private bool TryUseHttps()
24             {
25                 try
26                 {
27                     using var testClient = new HttpClient(
28                         new HttpClientHandler { ServerCertificateCustomValidationCallback = (a, b, c,
29                             ↪ d) => true });
30                     var response = testClient.GetAsync("https://localhost:7259/api/Product").Result;
31                     return response.IsSuccessStatusCode;
32                 }
33                 catch
34                 {
35                     return false;
36                 }
37             }
38
39             // === Методы работы с товарами ===
40
41             public async Task<List<Product>> GetProductsAsync() // Получить список товаров
```

```
41     {
42         try
43         {
44             return await _http.GetFromJsonAsync<List<Product>>("Product") ?? new
45                 ↳ List<Product>();
46         }
47         catch (Exception ex)
48         {
49             MessageBox.Show($"Ошибка при получении данных: {ex.Message}");
50             return new List<Product>();
51         }
52     }
53
54     public async Task AddProductAsync(Product product) => // Добавить новый товар
55         await _http.PostAsJsonAsync("Product", product);
56
57     public async Task UpdateProductAsync(Product product) => // Обновить товар по ID
58         await _http.PutAsJsonAsync($"Product/{product.Id}", product);
59
60     public async Task DeleteProductAsync(int id) => // Удалить товар по ID
61         await _http.DeleteAsync($"Product/{id}");
62
63     // === Методы работы с заказами ===
64
65     public async Task<List<Order>> GetOrdersAsync() // Получить список заказов
66     {
67         try
68         {
69             return await _http.GetFromJsonAsync<List<Order>>("Order") ?? new List<Order>();
70         }
71         catch (Exception ex)
72         {
73             MessageBox.Show($"Ошибка при получении заказов: {ex.Message}");
74             return new List<Order>();
75         }
76     }
77
78     public async Task AddOrderAsync(Order order) => // Добавить новый заказ
79         await _http.PostAsJsonAsync("Order", order);
80
81     public async Task DeleteOrderAsync(int id) => // Удалить заказ по ID
82         await _http.DeleteAsync($"Order/{id}");
83 }
```

Листинг 3.1 – Клиентского сервиса

Описание методов (к листингу 3.1):

1. `GetProductsAsync()` — выполняет GET-запрос на получение всех товаров.

4. Реализация MainViewModel.cs

MainViewModel.cs — основная ViewModel клиентского WPF-приложения. Класс MainViewModel реализует основную бизнес-логику WPF-клиента. Он управляет товарами, корзиной, заказами и взаимодействует с ApiService для выполнения операций через Web API. Все команды пользовательского интерфейса (добавление, удаление, обновление) реализованы через команды MVVM-паттерна. В листинге 4.1 представлены методы основной ViewModel разрабатываемого сервиса.

```
1 public class MainViewModel : INotifyPropertyChanged
2 {
3     private readonly ApiService _api = new(); // Экземпляр сервиса для обращения к Web API
4
5     public ObservableCollection<Product> Products { get; set; } = new(); // Список всех товаров
6     public ObservableCollection<Order> Orders { get; set; } = new(); // Список заказов
7     public ObservableCollection<CartItem> CartItems { get; set; } = new(); // Список товаров в
8     ↪ корзине
9
10    // Вводимые пользователем данные о товаре
11    public string NewProductName { get; set; } // Название
12    public decimal NewProductPrice { get; set; } // Цена
13    public int NewProductStock { get; set; } // Количество
14    public string SelectedCategory { get; set; } // Категория
15
16    // Данные покупателя для заказа
17    public string CustomerName { get; set; }
18
19    // Выбранные объекты UI
20    public Product SelectedProduct { get; set; }
21    public CartItem SelectedCartItem { get; set; }
22    public Order SelectedOrder { get; set; }
23
24    // Предустановленные категории для выбора
25    public ObservableCollection<string> Categories { get; } = new()
26    {
27        "Ноутбуки", "Комплектующие", "Аксессуары", "Мониторы", "Сеть"
28    };
29
30    // Команды для привязки к элементам UI
31    public ICommand AddProductCommand { get; }
32    public ICommand UpdateProductCommand { get; }
33    public ICommand DeleteProductCommand { get; }
34    public ICommand AddToCartCommand { get; }
35    public ICommand RemoveFromCartCommand { get; }
36    public ICommand AddOrderCommand { get; }
37    public ICommand DeleteOrderCommand { get; }
38    public ICommand ClearFieldsCommand { get; }
39
40    // Конструктор инициализирует команды и загружает данные
41    public MainViewModel()
```

```
41 {
42     AddProductCommand = new RelayCommand(async _ => await AddProductAsync(), _ =>
43         ↪ CanAddProduct());
44     UpdateProductCommand = new RelayCommand(async _ => await UpdateProductAsync(), _ =>
45         ↪ SelectedProduct != null);
46     DeleteProductCommand = new RelayCommand(async _ => await DeleteProductAsync(), _ =>
47         ↪ SelectedProduct != null);
48     AddToCartCommand = new RelayCommand(_ => AddToCart(), _ => SelectedProduct != null &&
49         ↪ SelectedProduct.Stock > 0);
50     RemoveFromCartCommand = new RelayCommand(_ => RemoveFromCart(), _ => SelectedCartItem !=
51         ↪ null);
52     AddOrderCommand = new RelayCommand(async _ => await AddOrderAsync(), _ => CartItems.Any()
53         ↪ && !string.IsNullOrEmpty(CustomerName));
54     DeleteOrderCommand = new RelayCommand(async _ => await DeleteOrderAsync(), _ =>
55         ↪ SelectedOrder != null);
56     ClearFieldsCommand = new RelayCommand(_ => ClearProductFields());
57
58     _ = LoadProductsAsync(); // Загрузка списка товаров
59     _ = LoadOrdersAsync();  // Загрузка списка заказов
60 }
61
62 private async Task LoadProductsAsync() // Получение всех товаров с сервера
63 {
64     Products.Clear();
65     var products = await _api.GetProductsAsync();
66     foreach (var p in products)
67         Products.Add(p);
68 }
69
70 private async Task LoadOrdersAsync() // Получение всех заказов с сервера
71 {
72     Orders.Clear();
73     var orders = await _api.GetOrdersAsync();
74     foreach (var order in orders)
75         Orders.Add(order);
76 }
77
78 private async Task AddProductAsync() // Добавление нового товара
79 {
80     var product = new Product
81     {
82         Name = NewProductName,
83         Price = NewProductPrice,
84         Stock = NewProductStock,
85         Category = SelectedCategory
86     };
87
88     await _api.AddProductAsync(product);
89     await LoadProductsAsync();
90     ClearProductFields();
91 }
```

```
85
86 private async Task UpdateProductAsync() // Обновление существующего товара
87 {
88     if (SelectedProduct == null) return;
89
90     SelectedProduct.Name = NewProductName;
91     SelectedProduct.Price = NewProductPrice;
92     SelectedProduct.Stock = NewProductStock;
93     SelectedProduct.Category = SelectedCategory;
94
95     await _api.UpdateProductAsync(SelectedProduct);
96     await LoadProductsAsync();
97     ClearProductFields();
98 }
99
100 private async Task DeleteProductAsync() // Удаление выбранного товара
101 {
102     if (SelectedProduct == null) return;
103     await _api.DeleteProductAsync(SelectedProduct.Id);
104     await LoadProductsAsync();
105     ClearProductFields();
106 }
107
108 private void AddToCart() // Добавление товара в корзину
109 {
110     var existing = CartItems.FirstOrDefault(x => x.Product.Id == SelectedProduct.Id);
111     if (existing != null)
112         existing.Quantity++;
113     else
114         CartItems.Add(new CartItem { Product = SelectedProduct, Quantity = 1 });
115
116     SelectedProduct.Stock--; // Уменьшаем остаток на складе
117 }
118
119 private void RemoveFromCart() // Удаление товара из корзины
120 {
121     if (SelectedCartItem == null) return;
122     SelectedCartItem.Product.Stock += SelectedCartItem.Quantity; // Возвращаем остаток
123     CartItems.Remove(SelectedCartItem);
124     SelectedCartItem = null;
125 }
126
127 private async Task AddOrderAsync() // Оформление нового заказа
128 {
129     var order = new Order
130     {
131         CustomerName = CustomerName,
132         OrderDate = DateTime.Now,
133         Items = CartItems.Select(c => new OrderItem
134         {
135             ProductId = c.Product.Id,
```

```
136         Quantity = c.Quantity
137     }).ToList()
138 };
139
140     await _api.AddOrderAsync(order);
141     await LoadOrdersAsync();
142
143     CartItems.Clear();
144     CustomerName = "";
145     await LoadProductsAsync(); // Обновление остатков
146 }
147
148 private async Task DeleteOrderAsync() // Удаление существующего заказа
149 {
150     if (SelectedOrder == null) return;
151     await _api.DeleteOrderAsync(SelectedOrder.Id);
152     await LoadOrdersAsync();
153 }
154
155 private void ClearProductFields() // Очистка полей формы ввода товара
156 {
157     NewProductName = "";
158     NewProductPrice = 0;
159     NewProductStock = 0;
160     SelectedCategory = null;
161 }
162
163 private bool CanAddProduct() => // Проверка валидности данных для добавления товара
164     !string.IsNullOrEmpty(NewProductName) &&
165     NewProductPrice > 0 &&
166     NewProductStock > 0 &&
167     !string.IsNullOrEmpty(SelectedCategory);
168
169 public event PropertyChangedEventHandler PropertyChanged; // Событие для INotifyPropertyChanged
170 protected void OnPropertyChanged([CallerMemberName] string name = null) =>
171     PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name)); // Уведомление об
    ↪ изменении свойства
172 }
```

Листинг 4.1 – MainViewModel.cs

Описание методов (к листингу 4.1):

1. LoadProductsAsync() — загружает список товаров с сервера.
2. LoadOrdersAsync() — загружает список заказов с сервера.
3. AddProductAsync() — формирует товар и отправляет на сервер.
4. UpdateProductAsync() — обновляет выделенный товар на сервере.

5. `DeleteProductAsync()` — удаляет выбранный товар.
6. `AddToCart()` — добавляет товар в корзину.
7. `RemoveFromCart()` — удаляет товар из корзины.
8. `AddOrderAsync()` — создает заказ на основе корзины и отправляет на сервер.
9. `DeleteOrderAsync()` — удаляет выбранный заказ.
10. `ClearProductFields()` — очищает форму ввода товара.
11. `CanAddProduct()` — проверяет, можно ли добавить товар.

Класс `MainViewModel` является связующим звеном между пользовательским интерфейсом и логикой работы с Web API. Все операции реализованы с учётом асинхронности и привязки данных.

5. OrderViewModel.cs — управление заказами

Класс `OrderViewModel` реализует логику взаимодействия с заказами. Он отвечает за загрузку существующих заказов, создание новых и удаление. Вся логика работает через `ApiService`, используя команды MVVM.

```
1 public class OrderViewModel : BaseViewModel
2 {
3     private readonly ApiService _api = new(); // Сервис API для обмена с сервером
4
5     private Order _selectedOrder; // Выбранный заказ в интерфейсе
6     private string _customerName; // Имя покупателя
7
8     public ObservableCollection<Order> Orders { get; } = new(); // Коллекция заказов
9     public ObservableCollection<Product> AvailableProducts { get; } = new(); // Доступные для
10    ↪ заказа товары
11
12    public ObservableCollection<OrderItem> SelectedItems { get; } = new(); // Позиции текущего
13    ↪ заказа
14
15    public string CustomerName
16    {
17        get => _customerName;
18        set => SetProperty(ref _customerName, value); // Установка имени покупателя
19    }
20
21    public Order SelectedOrder
22    {
23        get => _selectedOrder;
24        set
25        {
26            if (SetProperty(ref _selectedOrder, value) && value != null)
27            {
28                CustomerName = value.CustomerName; // Заполняем имя клиента из заказа
29                SelectedItems.Clear(); // Очищаем старые позиции
30                foreach (var item in value.Items)
31                {
32                    SelectedItems.Add(new OrderItem // Копируем позиции
33                    {
34                        ProductId = item.ProductId,
35                        Product = item.Product,
36                        Quantity = item.Quantity
37                    });
38                }
39            }
40        }
41    }
42
43    // Команды для кнопок интерфейса
44    public ICommand AddOrderCommand { get; }
45    public ICommand DeleteOrderCommand { get; }
```

```
43
44 // Конструктор: инициализация и загрузка данных
45 public OrderViewModel()
46 {
47     AddOrderCommand = new RelayCommand(async _ => await AddOrder());
48     DeleteOrderCommand = new RelayCommand(async _ => await DeleteOrder(), _ => SelectedOrder
49         ↪ != null);
50
51     _ = LoadOrders(); // Загрузка заказов
52     _ = LoadAvailableProducts(); // Загрузка товаров
53 }
54
55 private async Task LoadOrders() // Получение списка заказов
56 {
57     Orders.Clear();
58     var orders = await _api.GetOrdersAsync();
59     foreach (var order in orders)
60         Orders.Add(order);
61 }
62
63 private async Task LoadAvailableProducts() // Получение списка всех товаров
64 {
65     AvailableProducts.Clear();
66     var products = await _api.GetProductsAsync();
67     foreach (var p in products)
68         AvailableProducts.Add(p);
69 }
70
71 private async Task AddOrder() // Создание нового заказа
72 {
73     if (string.IsNullOrWhiteSpace(CustomerName) || SelectedItems.Count == 0) return;
74
75     var newOrder = new Order
76     {
77         CustomerName = CustomerName,
78         OrderDate = DateTime.Now,
79         Items = SelectedItems.Select(i => new OrderItem
80             {
81                 ProductId = i.Product.Id,
82                 Quantity = i.Quantity
83             }).ToList()
84     };
85
86     await _api.AddOrderAsync(newOrder); // Отправка заказа на сервер
87     await LoadOrders(); // Обновление списка
88
89     CustomerName = ""; // Очистка формы
90     SelectedItems.Clear();
91 }
92
93 private async Task DeleteOrder() // Удаление выбранного заказа
```

```
93     {  
94         if (SelectedOrder == null) return;  
95  
96         await _api.DeleteOrderAsync(SelectedOrder.Id);  
97         Orders.Remove(SelectedOrder);  
98         SelectedOrder = null;  
99         SelectedItems.Clear();  
100        CustomerName = string.Empty;  
101    }  
102 }
```

Листинг 5.1 – OrderViewModel.cs

Описание методов (к листингу 5.1):

1. LoadOrders() — загружает список всех заказов с сервера.
2. LoadAvailableProducts() — загружает товары для формирования нового заказа.
3. AddOrder() — создаёт новый заказ и отправляет его на сервер.
4. DeleteOrder() — удаляет выбранный заказ.

Класс OrderViewModel выполняет функции управления заказами в клиентской части и полностью опирается на Web API. Реализация соответствует шаблону MVVM и позволяет упростить обработку заказов в интерфейсе WPF.

6. Контрольные вопросы ?

1. В чём заключается основное отличие архитектуры лабораторной работы №6 от №4?
2. Какие преимущества даёт использование ASP.NET Core Web API по сравнению с прямым доступом к базе данных?
3. Как реализуется добавление товара в лабораторной №6? Чем оно отличается от предыдущей реализации?
4. Для чего используется класс ApiService? Какие методы он предоставляет?
5. Как происходит удаление заказа в WPF-приложении, работающем с Web API?
6. Почему необходимо использовать await при вызовах методов ApiService?
7. В каком формате передаются данные между клиентом и сервером?
8. Какие классы относятся к слою ViewModel, и как они связаны с моделью?
9. Как осуществляется привязка команд в XAML и зачем используются команды MVVM?
10. Что делает метод LoadOrders() в OrderViewModel и какие данные он загружает?
11. Какие изменения произошли в механизме сохранения заказов в новой архитектуре?
12. Почему важно разделение клиентской и серверной логики при разработке приложений?